

## SAE interpretability note: how many atoms does the next-track GRU surface, and why

*What this documents: the question “why does the model-SAE only detail ~8 atoms?”, the changes made to answer it (display cap, threshold, dictionary width, an L1 sweep, and a new top-k SAE mode), and what the numbers say. Analysis-only — nothing was promoted; the primary metric stays Recall@10.*

Model under the lens: `best-seq-nexttrack-20260718-230948-d3d4a` (the anti-eager next-track GRU, hidden width 256, Recall@10 0.157), analyzed on `seq-20260718-222807` via `POST /api/interp/model-sae`. The per-model SAE trains a sparse autoencoder over the GRU’s per-step hidden state and reads its atoms against the **next item** (artist / genre / album).

### The question

A run “details” only a handful of atoms (8–12) even though the dictionary has hundreds. Why?

Two independent causes, found by reading the engine (`predictors/seq_model_sae.py`):

1. **A hard display cap.** `atoms_by_concept` was the top **12** atoms by concept-separation (`shown = argsort(-best_sep)[:12]`), and only those that map to a concept (`best_field` is not `None`) are listed — so 8–12, never more, regardless of how many real concepts exist. `n_interpretable_concepts` (the *count* passing the bar) was already large (509), so the cap, not the bar, was what you saw.
2. **An under-sparsified code.** The SAE was barely sparse — **82% of atoms fire on every row** (`10_mean` 418 / 512). A dense code has no clean, monosemantic atoms; every atom weakly correlates with the most frequent next-artists, which is exactly what the detail list showed (Rammstein appeared 4× in the top 6).

### Changes made

| lever                      | where                                                     | before → after                    | intent                                                                      |
|----------------------------|-----------------------------------------------------------|-----------------------------------|-----------------------------------------------------------------------------|
| detail cap                 | <code>seq_model_sae.py</code><br><code>SHOWN_N</code>     | 12 → <b>40</b>                    | list more of the atoms that map to a concept                                |
| interpretability threshold | <code>seq_model_sae.py</code><br><code>CONCEPT_SEP</code> | 0.5 → <b>0.35</b>                 | count weaker-but-real concept atoms (aligns with <code>DROPPED_SEP</code> ) |
| dictionary width           | API <code>n_atoms</code>                                  | 0 (→512) → <b>2048</b>            | 4× the capacity for distinct atoms                                          |
| sparsity penalty           | API 11                                                    | 0.0015 → <b>0.05</b> → <b>0.5</b> | push the code toward a sparse, monosemantic dictionary                      |

The first two are predictor-code edits (Python; the server shells out to it, so no rebuild or restart). The last two are runtime API parameters.

## Results — one model, four SAE configs

| run                       | dict | L1     | l0_mean<br>(active)         | var_explained | n_interpretable       | dead | top atoms                  |
|---------------------------|------|--------|-----------------------------|---------------|-----------------------|------|----------------------------|
| baseline                  | 512  | 0.0015 | 418 / 512<br>(82%)          | 0.991         | 509                   | 12   | Rammstein<br>×4            |
| wide                      | 2048 | 0.0015 | 1291 / 2048<br>(63%)        | 0.995         | <b>2048<br/>(all)</b> | 40   | frequent<br>artists        |
| wide +<br>sparse          | 2048 | 0.05   | 1149 / 2048<br>(56%)        | 0.995         | 2048 (all)            | 40   | frequent<br>artists        |
| wide +<br>very-<br>sparse | 2048 | 0.5    | 715 / 2048<br>(35%)         | 0.988         | 1978                  | 40   | frequent<br>artists        |
| <b>top-k<br/>(k=32)</b>   | 2048 | —      | <b>32 / 2048<br/>(1.6%)</b> | 0.900         | 1351                  | 40   | <b>rare,<br/>selective</b> |

(utilization 1.0 and dead\_atoms 0 throughout — no atom goes unused across the population; top-k just caps how many fire *per row*.)

### How the changes reflect in the results

- **The cap change did what it should.** Every widened run now details **40** atoms instead of 8–12. That was the direct fix to the surface question.
- **Widening + lowering the threshold *saturates* the concept count.** At 2048 atoms with CONCEPT\_SEP 0.35, n\_interpretable\_concepts becomes **2048** — **every atom passes**. That is a red flag, not a win: when *all* atoms count as interpretable, the metric has stopped discriminating. So “more atoms” here is bookkeeping, not more genuine structure.
- **The code will not sparsify.** This is the load-bearing finding. Sweeping L1 over **330×** (0.0015 → 0.5) only moved the active fraction from **82% → 35%** — nowhere near a sparse dictionary (a real SAE wants a few percent active) — while reconstruction barely budged (var\_explained 0.99 the whole way). The GRU’s hidden state is dense and easy to reconstruct, so the L1 term trades off very slowly; the atoms stay **polysemantic frequent-artist detectors** (The Avalanches, Kraftwerk, DIIV, Rammstein, “album rock”) at every L1.
- **One real, partial effect of L1:** stronger sparsity makes the *individual* surfaced atoms cleaner even though the whole code stays dense — top-atom separation rose (assoc 3.8 → 4.9) and their firing rate fell (freq 0.85 → 0.35) from baseline to L1 0.5. So higher L1 sharpens atoms; it just doesn’t sparsify the population.

### The fix: a top-k SAE

Since the L1 penalty won’t drive the population sparse, I added a **top-k mode** to the engine (SAE(topk=k) in seq\_model\_sae.py, exposed as --topk on the predictor’s model-sae CLI). It keeps only the k largest ReLU activations per row and zeros the rest, so **l0 == k by construction** — no reliance on L1.

At **k = 32** (1.6% of the 2048-atom dictionary) the code is finally, genuinely sparse — and the atoms change character completely:

- **l0 = 32.0 exactly** (vs the L1 floor of 715).
- **Atoms become selective, not dense.** Firing rate collapses from 0.35–0.85 (L1) to **0.016–0.031** — each surfaced atom now activates on ~2% of steps and separates its concept sharply (assoc 5.1–5.6). The detail list spreads across more distinct artists/genres (Massive Attack, DIIIV, The xx, “indie rock”, ...) rather than repeating the few most frequent artists.
- **Honest cost:** reconstruction drops from `var_explained` ~0.99 to **0.90** — the sparsity fidelity trade a top-k SAE makes explicit. `n_interpretable` also falls off its ceiling (2048 → 1351), i.e. the metric starts discriminating again.

So top-k is the mechanism that actually turns this dense hidden state into a sparse dictionary of selective features; L1, dictionary width, and the threshold never could.

## Takeaways

1. The “8 atoms” was mostly a **display cap** (now 40), not a shortage of concepts — the count was already ~500.
2. On this recurrent hidden state, **n\_interpretable\_concepts is a saturated metric**: at any generous width/threshold it approaches the full dictionary, so it should not be read as “the model has N clean concepts.” Rely instead on the *ranked atoms\_by\_concept* and on *next\_item\_decodability* (which stayed informative: acc 0.42, AUC 0.79 vs a 0.007 baseline).
3. Getting a genuinely monosemantic dictionary needs a **different sparsity mechanism**, not a wider dictionary / lower threshold / bigger L1 knob. The **top-k SAE** added here does it: l0 is fixed directly, and at k=32 the atoms become sparse and selective (firing ~2% vs ~50%), at a reconstruction cost the method makes explicit (`var_explained` 0.90). Use top-k when per-atom interpretability is the goal; use L1 when faithful reconstruction matters more.

## Reproduce

*# The code levers are in predictors/seq\_model\_sae.py: SHOWN\_N=40, CONCEPT\_SEP=0.35, # and the SAE(topk=k) top-k mode. Run directly via the predictor CLI:*

```
predictors/.venv/bin/python predictors/seq_nexttrack.py model-sae \
--model data/models/best-seq-nexttrack-20260718-230948-d3d4a \
--dataset data/datasets/seq-20260718-222807 \
--output /tmp/topk32.json --n-atoms 2048 --topk 32 --epochs 40
```

*# Both modes also run through the server API (topk is now a first-class field):*

```
curl -s -X POST localhost:8096/api/interp/model-sae -H content-type:application/json \
-d '{"model":"best-seq-nexttrack-20260718-230948-d3d4a","n_atoms":2048,"topk":32,"epochs":40}
```

*(topk is wired end-to-end: ModelSaeRequest.topk in the server → --topk to the predictor. 0/absent L1-only, so existing calls are unchanged.)*